Universidad Nacional de Colombia

Programación Orientada a Objetos 2026-2

Actividad 2

Ejercicios de clases, atributos, métodos get/set y UML (basados en el libro de Leonardo Bermón Angarita)

Estudiante: Santiago Herrera Martínez

Docente: Walter Hugo Arboleda Mazo

Fecha de entrega: 8 de octubre de 2026

Repositorio GitHub: https://github.com/santiHM-cloud/actividad-2-POO

Ejercicio 2.1 — Definición de clases (clase Persona)

Enunciado

Se modela el concepto de una persona, con nombre, apellidos, número de documento de identidad y año de nacimiento. La clase tiene un constructor que inicializa los atributos y un método que imprime los datos en pantalla. En el main se crean dos personas y se muestran sus datos.

Código fuente (Java)

```
public class Persona {

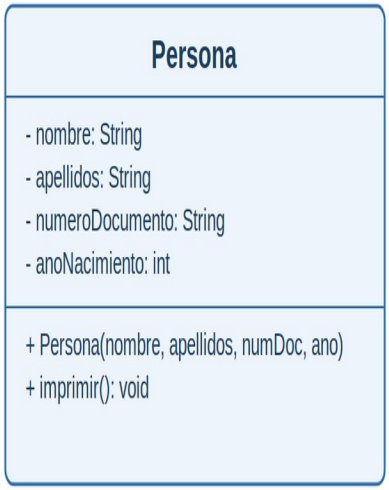
    private String nombre;
    private String apellidos;
    private String numeroDocumento;
    private int anoNacimiento;

    public Persona(String nombre, String apellidos, String numeroDocumento, int anoNacimiento) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.numeroDocumento = numeroDocumento;
        this.anoNacimiento = anoNacimiento;
    }

    public void imprimir() {
        System.out.println("Nombre: " + nombre);
        System.out.println("Apellidos: " + apellidos);
        System.out.println("Documento: " + numeroDocumento);
        System.out.println("Ano de nacimiento: " + anoNacimiento);
        System.out.println();
    }

    public static void main(String[] args) {
        Persona p1 = new Persona("Santiago", "Herrera Martinez", "1193001234", 2006);
        Persona p2 = new Persona("Maria", "Gomez Lopez", "1098765432", 1999);
        p1.imprimir();
        p2.imprimir();
    }
}
```

Diagrama de clases



Ejercicio 2.2 — Atributos con tipos primitivos y enumerados (clase Planeta)

Enunciado

Se modela un planeta del sistema solar, con nombre, cantidad de satélites, masa, volumen, diámetro, distancia al sol, tipo (enumerado: GASEOSO, TERRESTRE o ENANO) y si es observable a simple vista. Se calcula la densidad del planeta (masa entre volumen) y si es un planeta exterior (más allá del cinturón de asteroides, ubicado entre 2.1 y 3.4 UA).

Código fuente (Java)

```
public class Planeta {

    enum TipoPlaneta { GASEOSO, TERRESTRE, ENANO }

    private static final double UA_EN_KM = 149597870;

    private String nombre = null;
    private int cantidadSatelites = 0;
    private double masa = 0;
    private double volumen = 0;
    private int diametro = 0;
    private int distanciaSol = 0;
    private TipoPlaneta tipo;
    private boolean esObservable = false;

    public Planeta(String nombre, int cantidadSatelites, double masa, double volumen,
                   int diametro, int distanciaSol, TipoPlaneta tipo, boolean esObservable) {
        this.nombre = nombre;
        this.cantidadSatelites = cantidadSatelites;
        this.masa = masa;
        this.volumen = volumen;
        this.diametro = diametro;
        this.distanciaSol = distanciaSol;
        this.tipo = tipo;
        this.esObservable = esObservable;
    }

    public void imprimir() {
        System.out.println("Planeta: " + nombre);
        System.out.println("Satelites: " + cantidadSatelites);
        System.out.println("Masa: " + masa);
        System.out.println("Volumen: " + volumen);
        System.out.println("Diametro: " + diametro);
        System.out.println("Distancia al sol: " + distanciaSol);
        System.out.println("Tipo: " + tipo);
        System.out.println("Observable: " + esObservable);
    }

    public double calcularDensidad() {
        return masa / volumen;
    }

    public boolean esPlanetaExterior() {
        double limiteCinturon = UA_EN_KM * 3.4;
        return distanciaSol > limiteCinturon;
    }

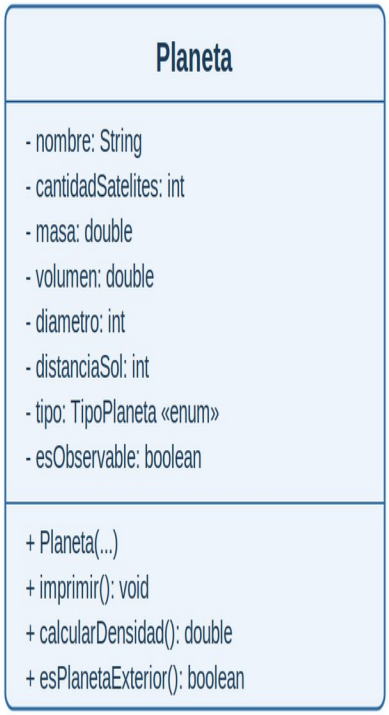
    public static void main(String[] args) {
        Planeta tierra = new Planeta("Tierra", 1, 5.9736E24, 1.08321E12, 12742,
                150000000, TipoPlaneta.TERRESTRE, true);
        Planeta jupiter = new Planeta("Jupiter", 79, 1.899E27, 1.4313E15, 139820,
                750000000, TipoPlaneta.GASEOSO, true);

        tierra.imprimir();
```

```
        System.out.println("Densidad: " + tierra.calcularDdensidad());
        System.out.println("Es exterior: " + tierra.esPlanetaExterior());
        System.out.println();

        jupiter.imprimir();
        System.out.println("Densidad: " + jupiter.calcularDdensidad());
        System.out.println("Es exterior: " + jupiter.esPlanetaExterior());
    }
}
```

Diagrama de clases



Ejercicio 2.3 — Estado de un objeto, métodos get y set (clase Automovil)

Enunciado

Se modela un automóvil con marca, modelo, motor, tipo de combustible, tipo de automóvil, número de puertas, cantidad de asientos, velocidad máxima, color y velocidad actual. Incluye métodos get y set, y métodos para acelerar y desacelerar (sin superar la velocidad máxima ni bajar de cero) y frenar. También calcula el tiempo estimado de llegada dada una distancia.

Código fuente (Java)

```
public class Automovil {

    enum TipoCombustible { GASOLINA, BIOETANOL, DIESEL, BIODIESEL, GAS_NATURAL }
    enum TipoAutomovil { CIUDAD, SUBCOMPACTO, COMPACTO, FAMILIAR, EJECUTIVO, SUV }
    enum Color { BLANCO, NEGRO, ROJO, NARANJA, AMARILLO, VERDE, AZUL, VIOLETA }

    private String marca;
    private int modelo;
    private int motor;
    private TipoCombustible tipoCombustible;
    private TipoAutomovil tipoAutomovil;
    private int numeroPuertas;
    private int cantidadAsientos;
    private int velocidadMaxima;
    private Color color;
    private int velocidadActual = 0;

    public Automovil(String marca, int modelo, int motor, TipoCombustible tipoCombustible,
                     TipoAutomovil tipoAutomovil, int numeroPuertas, int cantidadAsientos,
                     int velocidadMaxima, Color color) {
        this.marca = marca;
        this.modelo = modelo;
        this.motor = motor;
        this.tipoCombustible = tipoCombustible;
        this.tipoAutomovil = tipoAutomovil;
        this.numeroPuertas = numeroPuertas;
        this.cantidadAsientos = cantidadAsientos;
        this.velocidadMaxima = velocidadMaxima;
        this.color = color;
    }

    // Getters
    public String getMarca() { return marca; }
    public int getModelo() { return modelo; }
    public int getMotor() { return motor; }
    public TipoCombustible getTipoCombustible() { return tipoCombustible; }
    public TipoAutomovil getTipoAutomovil() { return tipoAutomovil; }
    public int getNumeroPuertas() { return numeroPuertas; }
    public int getCantidadAsientos() { return cantidadAsientos; }
    public int getVelocidadMaxima() { return velocidadMaxima; }
    public Color getColor() { return color; }
    public int getVelocidadActual() { return velocidadActual; }

    // Setters
    public void setVelocidadActual(int velocidadActual) { this.velocidadActual =
velocidadActual; }

    public void acelerar(int incremento) {
        if (velocidadActual + incremento > velocidadMaxima) {
            System.out.println("No se puede acelerar mas, se supera la velocidad maxima");
        } else {
            velocidadActual += incremento;
        }
    }
```

```
    public void desacelerar(int decremento) {
        if (velocidadActual - decremento < 0) {
            System.out.println("No se puede desacelerar, la velocidad quedaria negativa");
        } else {
            velocidadActual -= decremento;
        }
    }

    public void frenar() {
        velocidadActual = 0;
    }

    public double calcularTiempoLlegada(double distanciaKm) {
        return distanciaKm / velocidadActual;
    }

    public void imprimir() {
        System.out.println("Marca: " + marca + ", Modelo: " + modelo + ", Motor: " + motor +
"L");
        System.out.println("Combustible: " + tipoCombustible + ", Tipo: " + tipoAutomovil);
        System.out.println("Puertas: " + numeroPuertas + ", Asientos: " + cantidadAsientos);
        System.out.println("Velocidad maxima: " + velocidadMaxima + ", Color: " + color);
        System.out.println("Velocidad actual: " + velocidadActual);
    }

    public static void main(String[] args) {
        Automovil auto = new Automovil("Mazda", 2024, 2, TipoCombustible.GASOLINA,
                TipoAutomovil.SUV, 4, 5, 180, Color.AZUL);

        auto.setVelocidadActual(100);
        System.out.println("Velocidad actual: " + auto.getVelocidadActual());

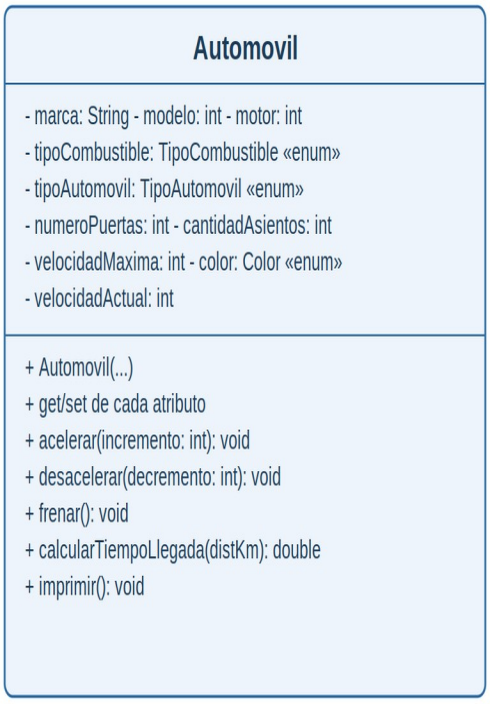
        auto.acelerar(20);
        System.out.println("Velocidad actual: " + auto.getVelocidadActual());

        auto.desacelerar(50);
        System.out.println("Velocidad actual: " + auto.getVelocidadActual());

        auto.frenar();
        System.out.println("Velocidad actual: " + auto.getVelocidadActual());

        auto.imprimir();
    }
}
```

Diagrama de clases


Automovil
- marca: String - modelo: int - motor: int
- tipoCombustible: TipoCombustible «enum»
- tipoAutomovil: TipoAutomovil «enum»
- numeroPuertas: int - cantidadAsientos: int
- velocidadMaxima: int - color: Color «enum»
- velocidadActual: int
+ Automovil(...)
+ get/set de cada atributo
+ acelerar(incremento: int): void
+ desacelerar(decremento: int): void
+ frenar(): void
+ calcularTiempoLlegada(distKm): double
+ imprimir(): void

Ejercicio 2.4 — Métodos con y sin valor de retorno (figuras geométricas)

Enunciado

Se modelan cuatro figuras geométricas: círculo, rectángulo, cuadrado y triángulo rectángulo, cada una con métodos para calcular su área y su perímetro. El triángulo rectángulo además calcula su hipotenusa y determina si es equilátero, isósceles o escaleno. Una clase de prueba (PruebaFiguras) contiene el método main que crea las cuatro figuras y prueba sus métodos.

Código fuente (Java)

```
public class Circulo {
    private double radio;

    public Circulo(double radio) {
        this.radio = radio;
    }

    public double calcularArea() {
        return Math.PI * Math.pow(radio, 2);
    }

    public double calcularPerimetro() {
        return 2 * Math.PI * radio;
    }
}
```

Clase Rectangulo.java

```
public class Rectangulo {
    private double base;
    private double altura;

    public Rectangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    public double calcularArea() {
        return base * altura;
    }

    public double calcularPerimetro() {
        return 2 * (base + altura);
    }
}
```

Clase Cuadrado.java

```
public class Cuadrado {
    private double lado;

    public Cuadrado(double lado) {
        this.lado = lado;
    }

    public double calcularArea() {
        return lado * lado;
    }

    public double calcularPerimetro() {
        return 4 * lado;
    }
}
```

Clase TrianguloRectangulo.java

```
public class TrianguloRectangulo {
    private double base;
    private double altura;

    public TrianguloRectangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    public double calcularArea() {
        return (base * altura) / 2;
    }

    public double calcularHipotenusa() {
        return Math.sqrt(Math.pow(base, 2) + Math.pow(altura, 2));
    }

    public double calcularPerimetro() {
        return base + altura + calcularHipotenusa();
    }

    public String determinarTipoTriangulo() {
        double hipotenusa = calcularHipotenusa();
        if (base == altura) {
            return "Isosceles (los catetos son iguales)";
        } else if (base != altura && base != hipotenusa && altura != hipotenusa) {
            return "Escaleno (todos los lados son diferentes)";
        } else {
            return "Isosceles";
        }
    }
}
```

Clase PruebaFiguras.java (main)

```
public class PruebaFiguras {
    public static void main(String[] args) {
        Circulo circulo = new Circulo(2);
        Rectangulo rectangulo = new Rectangulo(1, 2);
        Cuadrado cuadrado = new Cuadrado(3);
        TrianguloRectangulo triangulo = new TrianguloRectangulo(3, 4);

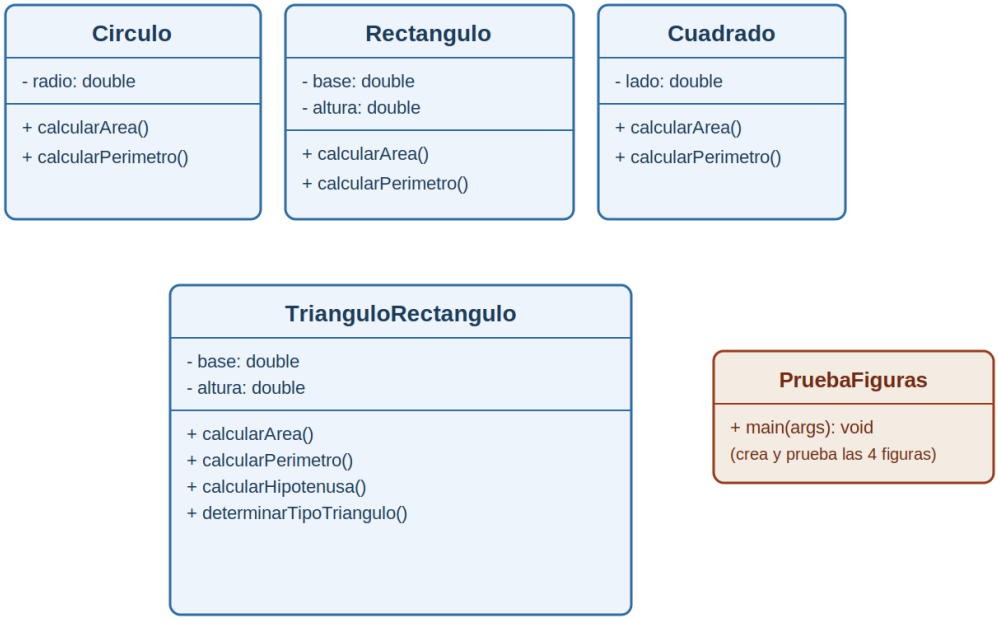
        System.out.println("Area del circulo: " + circulo.calcularArea());
        System.out.println("Perimetro del circulo: " + circulo.calcularPerimetro());
        System.out.println();

        System.out.println("Area del rectangulo: " + rectangulo.calcularArea());
        System.out.println("Perimetro del rectangulo: " + rectangulo.calcularPerimetro());
        System.out.println();

        System.out.println("Area del cuadrado: " + cuadrado.calcularArea());
        System.out.println("Perimetro del cuadrado: " + cuadrado.calcularPerimetro());
        System.out.println();

        System.out.println("Area del triangulo: " + triangulo.calcularArea());
        System.out.println("Hipotenusa: " + triangulo.calcularHipotenusa());
        System.out.println("Perimetro del triangulo: " + triangulo.calcularPerimetro());
        System.out.println("Tipo: " + triangulo.determinarTipoTriangulo());
    }
}
```

Diagrama de clases


Circulo
- radio: double
+ calcularArea()
+ calcularPerimetro()
Rectangulo
- base: double
- altura: double
+ calcularArea()
+ calcularPerimetro()
Cuadrado
- lado: double
+ calcularArea()
+ calcularPerimetro()
TrianguloRectangulo
- base: double
- altura: double
+ calcularArea()
+ calcularPerimetro()
+ calcularHipotenusa()
+ determinarTipoTriangulo()
PruebaFiguras
+ main(args): void
(crea y prueba las 4 figuras)

Ejercicio 2.5 — Métodos con parámetros (clase CuentaBancaria)

Enunciado

Se modela una cuenta bancaria con nombres y apellidos del titular, número de cuenta, tipo de cuenta (ahorros o corriente) y saldo (inicia en cero). Se pueden consultar el saldo, consignar un valor (aumenta el saldo) y retirar un valor (disminuye el saldo), validando que no se pueda retirar más de lo disponible.

Código fuente (Java)

```
public class CuentaBancaria {

    enum TipoCuenta { AHORROS, CORRIENTE }

    private String nombres;
    private String apellidos;
    private String numeroCuenta;
    private TipoCuenta tipoCuenta;
    private double saldo = 0;

    public CuentaBancaria(String nombres, String apellidos, String numeroCuenta, TipoCuenta
tipoCuenta) {
        this.nombres = nombres;
        this.apellidos = apellidos;
        this.numeroCuenta = numeroCuenta;
        this.tipoCuenta = tipoCuenta;
        this.saldo = 0;
    }

    public void imprimir() {
        System.out.println("Titular: " + nombres + " " + apellidos);
        System.out.println("Cuenta: " + numeroCuenta + " (" + tipoCuenta + ")");
        System.out.println("Saldo: " + saldo);
    }

    public double consultarSaldo() {
        return saldo;
    }

    public void consignar(double valor) {
        saldo += valor;
    }

    public void retirar(double valor) {
        if (valor > saldo) {
            System.out.println("No se puede retirar, el valor supera el saldo disponible");
        } else {
            saldo -= valor;
        }
    }

    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria("Santiago", "Herrera Martinez",
                "3001122334", TipoCuenta.AHORROS);

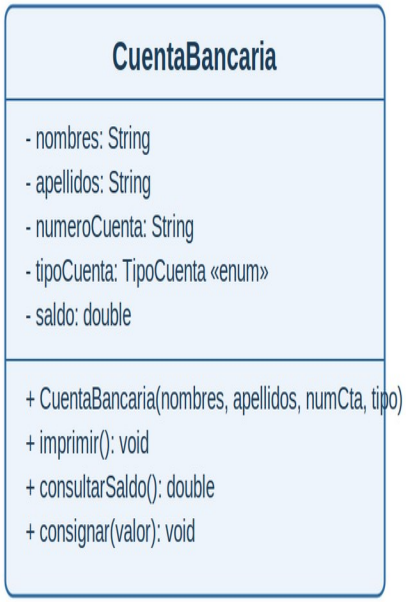
        cuenta.imprimir();

        cuenta.consignar(500000);
        System.out.println("Saldo tras consignar 500000: " + cuenta.consultarSaldo());

        cuenta.retirar(150000);
        System.out.println("Saldo tras retirar 150000: " + cuenta.consultarSaldo());

        cuenta.retirar(1000000);
        System.out.println("Saldo final: " + cuenta.consultarSaldo());
    }
}
```

Diagrama de clases



Repositorio del código: https://github.com/santiHM-cloud/actividad-2-POO